

HANBOOK

SQL

MÓNICA ORTIZ ORTIZ

LICENCIATURA EN ACTUARIA

SÉPTIMO SEMESTRE

UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

Contenido

CONCEPTOS BASES DE SQL	6
¿Qué es SQL?	6
¿Qué es una base de datos relacional?.....	6
Principales comandos SQL	6
SELECT: Consultar datos	6
INSERT: Insertar datos.....	7
UPDATE: Modificar datos	7
DELETE: Eliminar datos	7
Estructura de una tabla	7
Filtrar y ordenar datos	8
WHERE: Filtrar resultados	8
ORDER BY: Ordenar resultados.....	8
Relaciones entre tablas (JOINS)	8
Funciones útiles	9
Consultas Fundamentales con SQL	9
WHERE: Aplicación de Condiciones	9
Modificación de Datos con SQL	10
INSERT: Incorporar Nuevos Registros	10
UPDATE: Actualización de Información Existente	10
DELETE: Remover Información	10
INNER JOIN: Combinar Datos con Coincidencias	10
LEFT JOIN: Retener Todos los Datos de la Primera Tabla	11
Funciones de Agregación y Agrupación	11
Funciones de Agregación	11
Agrupación de Datos (GROUP BY)	12
Subconsultas	14
¿Cómo funcionan las subconsultas?	14
Tipos de subconsultas	14
Tipos de Comandos SQL: DDL, DML, DCL y TCL	17
DDL - Data Definition Language (Lenguaje de Definición de Datos)	17
DML - Data Manipulation Language (Lenguaje de Manipulación de Datos)	18
DCL - Data Control Language (Lenguaje de Control de Datos)	19

TCL - Transaction Control Language (Lenguaje de Control de Transacciones)	20
Tipos de Datos en SQL	21
1. Tipos de Datos Numéricos	21
2. Tipos de Datos de Texto	22
3. Tipos de Datos de Fecha y Hora	23
4. Tipos de Datos Booleanos	24
5. Tipos de Datos Binarios	25
Resumen de los Tipos de Datos Comunes en SQL	25
BETWEEN para Filtrar por Rango de Valores	26
Usando IN para Seleccionar Varias Opciones	26
Usando IS NULL para Filtrar Valores Nulos	27
Usando LIKE para Búsquedas con Patrones (Cadenas de Texto)	27
Creación de Tablas en SQL	29
Sintaxis Básica de CREATE TABLE	29
Actualizar Datos con SQL (UPDATE)	32
¿Cómo Funciona el Comando UPDATE?	32
Eliminar Datos en SQL: DROP vs. DELETE vs. TRUNCATE	36
DROP: Elimina Toda la Tabla u Otro Objeto de la Base de Datos	36
DELETE: Borra Filas Específicas de una Tabla	36
TRUNCATE: Borra Todos los Datos de una Tabla Rápidamente	37
¿Cuándo Usar Cada Uno?	38
Modificación de Tablas en SQL: ALTER TABLE	38
¿Qué Podemos Hacer con ALTER TABLE?	38
Sintaxis Básica	39
Operaciones Comunes con ALTER TABLE	39
1. Agregar una Columna a una Tabla	39
2. Eliminar una Columna	39
3. Modificar el Tipo de Datos de una Columna	40
4. Renombrar una Columna	40
5. Renombrar una Tabla	40
6. Agregar Restricciones (Constraints)	41
Agregar una Clave Foránea	41
Operaciones Comunes con Varias Tablas en SQL	41

Uso de JOIN para Combinar Tablas	42
LEFT JOIN (LEFT OUTER JOIN)	42
RIGHT JOIN (RIGHT OUTER JOIN)	43
FULL JOIN (FULL OUTER JOIN)	43
CROSS JOIN	44
Uso de UNION para Combinar Resultados de Consultas	44
¿Qué es UNION?	44
Expresiones SQL: CASE y COALESCE	45
CASE:	45
COALESCE:	45
Filtrando Datos Agrupados (HAVING).....	45
Diferencia entre WHERE y HAVING:	45
Cláusula OVER y Funciones de Ventana	46
Funciones de Ventana:	47
Clasificación de Registros Agrupados: RANK y ROW_NUMBER.....	48
Vistas (Views).....	48
Procedimientos Almacenados (Stored Procedures)	48
Disparadores (Triggers)	49
Administración de Bases de Datos en SQL	49
1. Crear una Base de Datos.....	50
¿Qué es una base de datos?	50
Comando para crear una base de datos:	50
2. Respalidar una Base de Datos	50
3. Restaurar una Base de Datos	51
Importar y Exportar Archivos	51
a) Importar datos con BULK INSERT	51
b) Exportar datos con bcp (Bulk Copy Program)	52
Consideraciones Finales para la Administración de Bases de Datos	52
Modelado de una Base de Datos Relacional en SQL.....	53
Elementos Clave del Modelado Relacional	53
1. Modelo Entidad-Relación (ER).....	53
2. Llaves (Primarias y Secundarias)	54
Llave Primaria (Primary Key).....	54

Llave Secundaria (Foreign Key)	54
3. Restricciones (Constraints)	55
4. Índices	56
5. Normalización	56
Optimización de Consultas SQL	58
Manejo de índices avanzados	59
Índices compuestos:	59
Índices únicos vs. no únicos:	59
Particionado de Tablas	59
Gestión de datos históricos	59
Seguridad y Control de Acceso	59
Gestión de usuarios y permisos	59
Cifrado de datos	60
Auditoría	60
Gestión Avanzada de Transacciones	60
Bloqueos y concurrencia	60
Manejo de deadlocks	61
Temas Relacionados con Big Data y SQL Moderno	61
Integración con sistemas NoSQL	61
SQL para analítica	61
SQL distribuido	61
Buenas Prácticas y Estilo de Escritura	62
Estandarización de consultas	62
Gestión de scripts	62
Gestión de entornos de desarrollo y producción	62
Herramientas Complementarias	62
Exploradores de bases de datos	62
Automatización con scripts	62
Consideraciones para Diferentes Motores de Bases de Datos	63
Diferencias entre SQL estándar y SQL en sistemas específicos	63

CONCEPTOS BASES DE SQL

¿Qué es SQL?

- **SQL (Structured Query Language):** Es un lenguaje estándar utilizado para interactuar con bases de datos relacionales.
- **Objetivo:** Permite realizar operaciones como:
 - **Consultar datos (SELECT).**
 - **Insertar datos (INSERT).**
 - **Actualizar datos (UPDATE).**
 - **Eliminar datos (DELETE).**

¿Qué es una base de datos relacional?

- Una **base de datos relacional** organiza la información en **tablas**
- Cada tabla tiene:
 - **Filas (registros):** Cada fila es una entrada única.
 - **Columnas (atributos):** Representan las características o propiedades de los datos.
- **Ejemplo:** Tabla clientes:

Identificador	Nombre	Edad
U37	Mónica Otiz	22
U38	Ebed Castillo	29

Principales comandos SQL

SELECT: Consultar datos

- Recupera información de una tabla.
- **Sintaxis básica:**
- `SELECT columna1, columna2 FROM nombre_tabla;`
- **Ejemplo:**
- `SELECT nombre, correo FROM clientes;`

INSERT: Insertar datos

- Agrega nuevos registros a una tabla.
- **Sintaxis básica:**
- INSERT INTO nombre_tabla (columna1, columna2) VALUES (valor1, valor2);
- **Ejemplo:**
- INSERT INTO clientes (id_cliente, nombre, correo)
- VALUES (3, 'Carlos', 'carlos@email.com');

UPDATE: Modificar datos

- Cambia datos existentes en una tabla.
- **Sintaxis básica:**
- UPDATE nombre_tabla
- SET columna1 = nuevo_valor1
- WHERE condición;
- **Ejemplo:**
- UPDATE clientes
- SET correo = 'nuevo@email.com'
- WHERE id_cliente = 1;

DELETE: Eliminar datos

- Borra registros de una tabla.
- **Sintaxis básica:**
- DELETE FROM nombre_tabla WHERE condición;
- **Ejemplo:**
- DELETE FROM clientes WHERE id_cliente = 2;

Estructura de una tabla

- **Crear una tabla:**
- CREATE TABLE nombre_tabla (
- columna1 tipo_dato restricción,
- columna2 tipo_dato restricción

-);
- **Tipos de datos más comunes:**
 - INT: Números enteros.
 - VARCHAR(n): Texto con un límite de n caracteres.
 - DATE: Fechas.
 - BOOLEAN: Verdadero o falso.
- **Ejemplo:**
- CREATE TABLE productos (
- id_producto INT PRIMARY KEY,
- nombre VARCHAR(100),
- precio DECIMAL(10, 2)
-);

Filtrar y ordenar datos

WHERE: Filtrar resultados

- Muestra solo los datos que cumplen una condición.
- **Ejemplo:**
- SELECT nombre, correo
- FROM clientes
- WHERE id_cliente = 1;

ORDER BY: Ordenar resultados

- Ordena los datos de manera ascendente (ASC) o descendente (DESC).
- **Ejemplo:**
- SELECT nombre, correo
- FROM clientes
- ORDER BY nombre ASC;

Relaciones entre tablas (JOINS)

- Combina datos de dos o más tablas basadas en una columna común.

- **Tipos de JOIN:**
 - **INNER JOIN:** Devuelve coincidencias en ambas tablas.
 - **LEFT JOIN:** Devuelve todos los registros de la tabla izquierda y las coincidencias de la derecha.

Funciones útiles

- **COUNT:** Cuenta filas.
- `SELECT COUNT(*) FROM clientes;`
- **SUM:** Suma valores de una columna.
- `SELECT SUM(precio) FROM productos;`
- **AVG:** Calcula el promedio.
- `SELECT AVG(precio) FROM productos;`

Consultas Fundamentales con SQL

SELECT: Recuperación de Datos

La instrucción SELECT se emplea para obtener información de una base de datos. Permite seleccionar columnas específicas o todas las columnas de una tabla.

Sintaxis general:

`SELECT columna1, columna2 FROM tabla;`

Ejemplo:

`SELECT nombre, correo FROM clientes;`

Este comando extraerá las columnas **nombre** y **correo** de la tabla **clientes**.

WHERE: Aplicación de Condiciones

La cláusula WHERE permite filtrar los datos según una o varias condiciones específicas.

Sintaxis general:

`SELECT columna1, columna2 FROM tabla WHERE condición;`

Ejemplo:

`SELECT nombre, correo FROM clientes WHERE id_cliente = 1;`

Este comando retornará los datos del cliente cuyo identificador (**id_cliente**) sea igual a 1.

Modificación de Datos con SQL

INSERT: Incorporar Nuevos Registros

El comando INSERT INTO sirve para agregar información nueva a una tabla.

Estructura básica:

```
INSERT INTO tabla (columna1, columna2) VALUES (valor1, valor2);
```

Ejemplo:

```
INSERT INTO clientes (id_cliente, nombre, correo) VALUES (3, 'Carlos', 'carlos@email.com');
```

Este comando añadirá un cliente con **id_cliente = 3** a la tabla **clientes**.

UPDATE: Actualización de Información Existente

El comando UPDATE permite modificar datos ya presentes en una tabla.

Estructura básica:

```
UPDATE tabla SET columna1 = valor1, columna2 = valor2 WHERE condición;
```

Ejemplo:

```
UPDATE clientes SET correo = 'nuevo_email@email.com' WHERE id_cliente = 1;
```

Este comando actualizará el correo del cliente con **id_cliente = 1**.

DELETE: Remover Información

El comando DELETE se utiliza para eliminar registros específicos de una tabla.

Estructura básica:

```
DELETE FROM tabla WHERE condición;
```

Ejemplo:

```
DELETE FROM clientes WHERE id_cliente = 3;
```

Este comando eliminará el cliente cuyo **id_cliente = 3** de la tabla **clientes**.

Relaciones entre Tablas (JOINS)

INNER JOIN: Combinar Datos con Coincidencias

El INNER JOIN devuelve las filas que tienen coincidencias en ambas tablas relacionadas.

Estructura básica:

```
SELECT columnas FROM tabla1 INNER JOIN tabla2 ON tabla1.columna =  
tabla2.columna;
```

Ejemplo:

```
SELECT clientes.nombre, pedidos.fecha FROM clientes INNER JOIN pedidos ON  
clientes.id_cliente = pedidos.id_cliente;
```

Este comando lista los nombres de los clientes junto con las fechas de sus pedidos.

LEFT JOIN: Retener Todos los Datos de la Primera Tabla

El LEFT JOIN incluye todas las filas de la primera tabla y, cuando no hay coincidencias en la segunda, retorna valores **NULL**.

Estructura básica:

```
SELECT columnas FROM tabla1 LEFT JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

Ejemplo:

```
SELECT clientes.nombre, pedidos.fecha FROM clientes LEFT JOIN pedidos ON  
clientes.id_cliente = pedidos.id_cliente;
```

Este comando devuelve todos los clientes, incluyendo aquellos sin pedidos.

Funciones de Agregación y Agrupación

Las funciones de agregación y agrupación en SQL son herramientas poderosas para realizar cálculos y resúmenes sobre grandes conjuntos de datos.

Funciones de Agregación

Las funciones de agregación permiten realizar cálculos en un conjunto de datos. Algunas de las funciones más comunes son:

1. COUNT

Cuenta el número de filas o registros que cumplen con una condición.

Ejemplo:

2. SELECT COUNT(*) FROM clientes;

Este comando devuelve el total de registros (clientes) en la tabla clientes. La función COUNT(*) cuenta todas las filas, sin importar si contienen valores nulos.

3. SUM

Suma los valores de una columna numérica.

Ejemplo:

4. SELECT SUM(total) FROM ventas;

Esto sumará todos los valores de la columna total en la tabla ventas. Es útil para calcular el total de ventas, por ejemplo.

5. AVG

Calcula el promedio de los valores en una columna numérica.

Ejemplo:

6. SELECT AVG(precio) FROM productos;

Este comando devuelve el promedio de los precios en la columna precio de la tabla productos.

7. MAX

Encuentra el valor máximo de una columna numérica.

Ejemplo:

8. SELECT MAX(salario) FROM empleados;

Esto devolverá el salario más alto de la tabla empleados.

9. MIN

Encuentra el valor mínimo de una columna numérica.

Ejemplo:

10. SELECT MIN(fecha) FROM pedidos;

Este comando devolverá la fecha más temprana en la columna fecha de la tabla pedidos.

Agrupación de Datos (GROUP BY)

La cláusula GROUP BY se utiliza junto con las funciones de agregación para agrupar los resultados en función de una o más columnas. Esto permite realizar cálculos sobre grupos de datos en lugar de sobre todos los registros.

Ejemplo 1: Agrupar por una columna

Supongamos que queremos contar cuántos clientes hay en cada ciudad. Para esto, usaríamos GROUP BY en la columna ciudad y aplicaríamos la función COUNT para contar los clientes de cada ciudad.

```
SELECT ciudad, COUNT(*)
```

```
FROM clientes
```

```
GROUP BY ciudad;
```

Explicación:

- La cláusula GROUP BY ciudad agrupa a todos los clientes por ciudad.

- La función COUNT(*) cuenta el número de clientes en cada grupo (en este caso, por ciudad).

El resultado será algo como esto:

Ciudad	Número de Clientes
--------	--------------------

Madrid	10
--------	----

Barcelona	8
-----------	---

Valencia	5
----------	---

Ejemplo 2: Agrupar por múltiples columnas

Podemos agrupar por más de una columna. Por ejemplo, si queremos contar los pedidos por cliente y por año:

```
SELECT id_cliente, YEAR(fecha_pedido) AS año, COUNT(*)
```

```
FROM pedidos
```

```
GROUP BY id_cliente, YEAR(fecha_pedido);
```

Explicación:

- Aquí estamos agrupando los pedidos primero por id_cliente y luego por el año en que se hizo el pedido (usando la función YEAR() para extraer el año de la fecha).
- La función COUNT(*) cuenta el número de pedidos en cada grupo.

Ordenación de los resultados

Además de agrupar los datos, a menudo es útil ordenar los resultados con la cláusula ORDER BY:

```
SELECT ciudad, COUNT(*)
```

```
FROM clientes
```

```
GROUP BY ciudad
```

```
ORDER BY COUNT(*) DESC;
```

Explicación:

- Aquí estamos ordenando los resultados de mayor a menor número de clientes por ciudad, usando ORDER BY COUNT(*) DESC.

El resultado será algo como esto:

Ciudad Número de Clientes

Madrid 10

Barcelona 8

Valencia 5

Subconsultas

Las **subconsultas** en SQL son consultas dentro de otras consultas. Básicamente, una subconsulta es una instrucción SELECT anidada dentro de otra consulta, y su propósito es devolver un resultado que será utilizado por la consulta principal.

¿Cómo funcionan las subconsultas?

Las subconsultas permiten realizar consultas más complejas, donde los resultados de una consulta interna (subconsulta) son utilizados por la consulta externa. Las subconsultas pueden devolver un único valor, un conjunto de valores, o una tabla completa.

Existen varios tipos de subconsultas, dependiendo de su ubicación y el tipo de resultado que devuelvan.

Tipos de subconsultas

1. **Subconsulta escalar (un solo valor)** Una subconsulta escalar devuelve un solo valor, que luego es utilizado por la consulta principal.

Ejemplo: Supongamos que queremos encontrar el nombre del cliente que hizo el pedido con el valor más alto.

```
SELECT nombre
```

```
FROM clientes
```

```
WHERE id_cliente = (
```

```
    SELECT id_cliente
```

```
    FROM pedidos
```

```
    WHERE total = (SELECT MAX(total) FROM pedidos)
```

```
);
```

Explicación:

- La **subconsulta interna** SELECT MAX(total) FROM pedidos devuelve el valor más alto de la columna **total**.

- La **subconsulta intermedia** `SELECT id_cliente FROM pedidos WHERE total = (...)` devuelve el **id_cliente** del pedido que tiene el valor máximo.
- Finalmente, la consulta principal obtiene el **nombre** del cliente con ese **id_cliente**.

2. Subconsulta en la cláusula **WHERE**

Las subconsultas se utilizan comúnmente en la cláusula **WHERE** para filtrar resultados según una condición derivada de otra consulta.

Ejemplo:

```
SELECT nombre, correo
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM pedidos
    WHERE fecha = '2024-11-01'
);
```

Explicación:

- La subconsulta devuelve todos los **id_cliente** de los pedidos realizados el 1 de noviembre de 2024.
- La consulta principal obtiene los **nombre** y **correo** de todos los clientes cuyo **id_cliente** está en esa lista de resultados.

3. Subconsulta en la cláusula **FROM**

Las subconsultas también pueden estar en la cláusula **FROM**, donde se comportan como una tabla derivada o "tabla temporal". Esto es útil cuando necesitas trabajar con un conjunto de resultados antes de realizar más operaciones sobre ellos.

Ejemplo:

```
SELECT promedio
FROM (
    SELECT AVG(total) AS promedio
    FROM pedidos
    WHERE fecha = '2024-11-01'
```

) AS subconsulta;

Explicación:

- La subconsulta dentro de **FROM** calcula el promedio de **total** de los pedidos realizados en una fecha específica.
- La consulta principal devuelve ese valor promedio calculado.

4. Subconsulta en la cláusula SELECT

También puedes usar subconsultas dentro de la cláusula **SELECT** para calcular valores agregados o realizar una consulta secundaria para cada fila de la consulta principal.

Ejemplo:

```
SELECT nombre, (  
    SELECT COUNT(*)  
    FROM pedidos  
    WHERE id_cliente = clientes.id_cliente  
) AS total_pedidos  
FROM clientes;
```

Explicación:

- Para cada fila en la tabla **clientes**, la subconsulta cuenta cuántos pedidos tiene ese cliente específico.
- La consulta principal devuelve el nombre del cliente junto con el número de pedidos.

Consideraciones importantes sobre las subconsultas

1. Subconsultas correlacionadas vs no correlacionadas:

- **Subconsultas no correlacionadas:** Son independientes de la consulta principal. Su resultado no depende de las filas de la consulta externa.
- **Subconsultas correlacionadas:** Están relacionadas con la consulta externa. Dependiendo de cada fila de la consulta principal, la subconsulta se evalúa nuevamente. Esto puede ser más costoso en términos de rendimiento.

Ejemplo de subconsulta correlacionada:

```
SELECT nombre
```

```
FROM clientes c
WHERE EXISTS (
    SELECT 1
    FROM pedidos p
    WHERE p.id_cliente = c.id_cliente
    AND p.total > 100
);
```

En este ejemplo, la subconsulta se ejecuta para cada fila de la tabla **clientes**, verificando si el cliente tiene un pedido con un **total** superior a 100.

2. **Rendimiento:** Las subconsultas pueden ser menos eficientes que las uniones (JOINS) en algunos casos, especialmente si son correlacionadas y se ejecutan para cada fila de la consulta principal. Dependiendo del motor de base de datos y el tipo de subconsulta, el rendimiento puede verse afectado.

Tipos de Comandos SQL: DDL, DML, DCL y TCL

En SQL, los comandos se agrupan en varias categorías, cada una de las cuales tiene un propósito específico en la gestión de bases de datos. Estas categorías son: **DDL**, **DML**, **DCL** y **TCL**.

DDL - Data Definition Language (Lenguaje de Definición de Datos)

Los **comandos DDL** se encargan de definir y modificar la estructura de la base de datos. Estos comandos afectan directamente la estructura de la base de datos, como la creación o eliminación de tablas, vistas e índices. Los cambios realizados con los comandos DDL son **permanentes**, lo que significa que una vez ejecutados, modifican de forma definitiva la estructura de la base de datos.

Principales comandos DDL:

- **CREATE:** Este comando se utiliza para crear nuevos objetos dentro de la base de datos, como tablas, índices o vistas.
 - **Ejemplo:** Crear una nueva tabla llamada "empleados".
- CREATE TABLE empleados (
 - id_empleado INT PRIMARY KEY,
 - nombre VARCHAR(100),

- salario DECIMAL(10, 2)
-);
- **ALTER**: Permite modificar la estructura de una tabla existente, como agregar, modificar o eliminar columnas.
 - **Ejemplo**: Agregar una nueva columna llamada **fecha_ingreso** a la tabla "empleados".
- ALTER TABLE empleados ADD fecha_ingreso DATE;
- **DROP**: Elimina un objeto de la base de datos, como una tabla, vista o índice.
 - **Ejemplo**: Eliminar la tabla "empleados".
- DROP TABLE empleados;
- **TRUNCATE**: Elimina todos los registros de una tabla, pero mantiene su estructura intacta para futuras inserciones. A diferencia de **DELETE**, no registra cada eliminación de fila.
 - **Ejemplo**: Eliminar todos los registros de la tabla "empleados" sin eliminar la tabla.
- TRUNCATE TABLE empleados;
- **RENAME**: Cambia el nombre de un objeto de la base de datos.
 - **Ejemplo**: Renombrar la tabla "empleados" a "personal".
- RENAME TABLE empleados TO personal;

DML - Data Manipulation Language (Lenguaje de Manipulación de Datos)

Los **comandos DML** se utilizan para manipular los **datos** almacenados en las tablas. Esto incluye la inserción, actualización, eliminación y consulta de datos. Los cambios realizados con los comandos DML afectan **solo los datos**, no la estructura de la base de datos, y son **temporales** (es decir, se pueden deshacer si no se han confirmado con un COMMIT).

Principales comandos DML:

- **SELECT**: Recupera datos de una o más tablas de la base de datos.
 - **Ejemplo**: Consultar todos los empleados de la tabla "empleados".
- SELECT id_employado, nombre, salario FROM empleados;
- **INSERT**: Inserta nuevos registros en una tabla.

- **Ejemplo:** Insertar un nuevo empleado en la tabla "empleados".
- INSERT INTO empleados (id_empleado, nombre, salario)
- VALUES (1, 'Juan Pérez', 3000.00);
- **UPDATE:** Modifica los datos existentes en una tabla.
 - **Ejemplo:** Actualizar el salario de un empleado con **id_empleado** igual a 1.
- UPDATE empleados
- SET salario = 3500.00
- WHERE id_empleado = 1;
- **DELETE:** Elimina registros existentes de una tabla.
 - **Ejemplo:** Eliminar el empleado con **id_empleado** igual a 1.
- DELETE FROM empleados WHERE id_empleado = 1;

DCL - Data Control Language (Lenguaje de Control de Datos)

Los **comandos DCL** se utilizan para controlar el **acceso** a los datos y gestionar los **permisos** de los usuarios dentro de la base de datos. Estos comandos son importantes para asegurar que solo los usuarios autorizados puedan realizar ciertas operaciones sobre los datos.

Principales comandos DCL:

- **GRANT:** Otorga permisos a los usuarios sobre una base de datos o sus objetos (tablas, vistas, etc.).
 - **Ejemplo:** Otorgar permisos de lectura (SELECT) y escritura (INSERT, UPDATE) sobre la tabla "empleados" a un usuario llamado "usuario1".
- GRANT SELECT, INSERT, UPDATE ON empleados TO usuario1;
- **REVOKE:** Revoca permisos previamente otorgados a los usuarios.
 - **Ejemplo:** Revocar el permiso de actualización (UPDATE) sobre la tabla "empleados" para el usuario "usuario1".
- REVOKE UPDATE ON empleados FROM usuario1;
- **DENY:** Niega explícitamente ciertos permisos a los usuarios. Este comando es más restrictivo que **REVOKE**.

- **Ejemplo:** Negar permisos de eliminación (DELETE) sobre la tabla "empleados" a un usuario.
- DENY DELETE ON empleados TO usuario1;

TCL - Transaction Control Language (Lenguaje de Control de Transacciones)

Los **comandos TCL** son utilizados para gestionar **transacciones** en la base de datos. Una transacción es un conjunto de operaciones SQL que se agrupan y se ejecutan de manera atómica: o todas se ejecutan, o ninguna lo hace. Los comandos TCL permiten controlar el inicio, la confirmación y la reversión de las transacciones, lo que ayuda a mantener la integridad de los datos.

Principales comandos TCL:

- **COMMIT:** Confirma una transacción, haciendo permanentes todos los cambios realizados desde el último **COMMIT**.
 - **Ejemplo:** Confirmar la transacción después de realizar cambios.
- COMMIT;
- **ROLLBACK:** Revierte todos los cambios realizados en la transacción desde el último **COMMIT** o desde un **SAVEPOINT**. Es útil para deshacer cambios cuando ocurre un error.
 - **Ejemplo:** Deshacer todos los cambios realizados en una transacción.
- ROLLBACK;
- **SAVEPOINT:** Establece un punto de control dentro de una transacción. Se puede usar para hacer un **ROLLBACK** solo a este punto, sin afectar el resto de la transacción.
 - **Ejemplo:** Establecer un punto de guardado dentro de una transacción.
- SAVEPOINT punto_guardado;
- **SET TRANSACTION:** Establece las propiedades de la transacción, como el nivel de aislamiento (cómo se gestionan los bloqueos y el acceso concurrente).
 - **Ejemplo:** Establecer el nivel de aislamiento de la transacción a **SERIALIZABLE** (el nivel más alto de aislamiento).

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Cada tipo de comando SQL tiene un propósito específico:

- **DDL** para estructurar la base de datos.
- **DML** para manipular los datos.
- **DCL** para controlar el acceso y la seguridad.
- **TCL** para manejar las transacciones y mantener la integridad de los datos.

Tipos de Datos en SQL

Cuando trabajamos con bases de datos, uno de los aspectos más importantes es definir correctamente los **tipos de datos** para cada columna en nuestras tablas. Los tipos de datos determinan qué tipo de valor puede almacenar una columna, y si no se eligen adecuadamente, podemos tener problemas de integridad, como intentar guardar texto donde se esperaba un número o perder precisión en cálculos matemáticos.

Los tipos de datos pueden variar ligeramente dependiendo del **Sistema de Gestión de Bases de Datos (SGBD)** que estés utilizando (por ejemplo, MySQL, SQL Server, PostgreSQL, etc.), pero la mayoría de ellos son similares.

1. Tipos de Datos Numéricos

Los **tipos numéricos** se usan para almacenar **números** en diferentes formatos. Dependiendo de si necesitas trabajar con **enteros** o con **números decimales**, utilizarás diferentes tipos de datos.

a. INT (Entero)

El tipo **INT** es uno de los más comunes para almacenar **números enteros**, es decir, aquellos que no tienen decimales. Se usa, por ejemplo, para almacenar el ID de un usuario o de un producto.

- **Ejemplo:** Crear una tabla para almacenar usuarios, donde **id** es un número entero.

```
CREATE TABLE usuarios (
    id INT PRIMARY KEY,
    nombre VARCHAR(100)
);
```

b. DECIMAL o NUMERIC

Estos tipos de datos se usan para almacenar números con **decimales**, como precios, salarios o cualquier valor que requiera una precisión específica en los decimales. Se pueden definir con una precisión específica, indicando cuántos **dígitos en total** y cuántos **decimales** tendrá el número.

- **Ejemplo:** Crear una tabla de productos con precios que tienen hasta 2 decimales.

```
CREATE TABLE productos (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    precio DECIMAL(10, 2) -- 10 dígitos en total, 2 después del punto decimal  
);
```

c. FLOAT / DOUBLE

FLOAT y **DOUBLE** también se usan para almacenar números con decimales, pero tienen diferente **precisión**. **FLOAT** tiene menor precisión, mientras que **DOUBLE** ofrece más precisión y es útil cuando se necesitan trabajar con **números muy grandes o pequeños** (por ejemplo, en cálculos científicos).

- **Ejemplo:** Usar **FLOAT** para almacenar una distancia con decimales.

```
CREATE TABLE medidas (  
    id INT PRIMARY KEY,  
    distancia FLOAT  
);
```

2. Tipos de Datos de Texto

Los **tipos de datos de texto** se utilizan para almacenar **cadenas de caracteres**, como nombres, direcciones, descripciones, etc. Son útiles cuando necesitas trabajar con **información alfanumérica**.

a. VARCHAR

VARCHAR es el tipo de dato de texto más usado. Se utiliza para almacenar cadenas de texto **de longitud variable**, es decir, ocupa solo el espacio necesario para almacenar los caracteres, no siempre el mismo tamaño. Puedes especificar un **número máximo de caracteres** para la cadena.

- **Ejemplo:** Crear una tabla para almacenar clientes, donde el nombre puede tener hasta 100 caracteres.

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100) -- El nombre puede tener hasta 100 caracteres
```

);

b. CHAR

A diferencia de **VARCHAR**, el tipo **CHAR** tiene una **longitud fija**, lo que significa que siempre ocupará el mismo espacio, incluso si el texto es más corto. Este tipo se usa cuando sabes que la longitud del texto será **siempre la misma** (por ejemplo, en códigos postales o códigos de país).

- **Ejemplo:** Crear una tabla para almacenar clientes con un código de país de 2 caracteres.

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    codigo_pais CHAR(2) -- El código de país siempre tendrá 2 caracteres  
);
```

c. TEXT

El tipo **TEXT** se usa para almacenar cadenas de texto **muy largas**, como descripciones, artículos o comentarios. Este tipo es útil cuando el tamaño de la cadena de texto es variable y puede superar el límite de **VARCHAR**.

- **Ejemplo:** Crear una tabla de productos con una descripción larga.

```
CREATE TABLE productos (  
    id INT PRIMARY KEY,  
    descripcion TEXT -- Descripción extensa del producto  
);
```

3. Tipos de Datos de Fecha y Hora

Cuando necesitas almacenar información sobre fechas y horas, como la **fecha de nacimiento de un usuario** o el **momento de realización de un pedido**, usarás tipos de datos de **fecha y hora**.

a. DATE

El tipo **DATE** se usa para almacenar **solo la fecha** (año, mes y día) sin la hora.

- **Ejemplo:** Crear una tabla de usuarios con la fecha de nacimiento.

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,
```

nombre VARCHAR(100),

fecha_nacimiento DATE -- Solo la fecha, sin la hora

);

b. DATETIME / TIMESTAMP

Los tipos **DATETIME** y **TIMESTAMP** se usan para almacenar tanto **la fecha como la hora** (año, mes, día, hora, minuto, segundo). La diferencia es que **TIMESTAMP** es más específico y puede almacenar también la **zona horaria** en algunos sistemas de bases de datos.

- **Ejemplo:** Crear una tabla de pedidos con la fecha y hora de realización.

```
CREATE TABLE pedidos (
```

```
    id INT PRIMARY KEY,
```

```
    id_cliente INT,
```

```
    fecha_pedido DATETIME -- Fecha y hora de la realización del pedido
```

```
);
```

c. TIME

El tipo **TIME** se usa para almacenar **solo la hora**, sin la fecha.

- **Ejemplo:** Crear una tabla para la tienda con la hora de apertura.

```
CREATE TABLE tienda (
```

```
    id INT PRIMARY KEY,
```

```
    hora_apertura TIME -- Solo la hora, sin la fecha
```

```
);
```

4. Tipos de Datos Booleanos

Los tipos **booleanos** se usan para almacenar valores de **verdadero o falso**. En algunos sistemas de bases de datos, como MySQL, puedes usar **TINYINT** (0 o 1) en lugar de **BOOLEAN** o **BOOL**.

a. BOOLEAN / BOOL

El tipo **BOOLEAN** almacena valores **TRUE** o **FALSE**.

- **Ejemplo:** Crear una tabla para productos, donde se indique si están disponibles o no.

```
CREATE TABLE productos (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    disponible BOOLEAN -- TRUE o FALSE, indicando si el producto está disponible  
);
```

5. Tipos de Datos Binarios

Los **tipos de datos binarios** se usan para almacenar **datos no textuales**, como imágenes, videos o archivos en formato binario.

a. BLOB

BLOB (Binary Large Object) es el tipo más común para almacenar **grandes cantidades de datos binarios**, como imágenes, audios o videos.

- **Ejemplo:** Crear una tabla para usuarios con una columna para almacenar la imagen de perfil.

```
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    foto_perfil BLOB -- Almacenar la imagen de perfil del usuario  
);
```

Resumen de los Tipos de Datos Comunes en SQL

- **Numéricos:** INT, DECIMAL, FLOAT, DOUBLE.
- **Texto:** VARCHAR, CHAR, TEXT.
- **Fecha y Hora:** DATE, DATETIME, TIME, TIMESTAMP.
- **Booleanos:** BOOLEAN, BOOL.
- **Binarios:** BLOB.

Es importante elegir el tipo de dato adecuado para cada columna en función del tipo de información que deseas almacenar, ya que esto garantiza la **integridad** de la base de datos y evita errores al manipular los datos.

BETWEEN para Filtrar por Rango de Valores

El operador **BETWEEN** se usa cuando necesitamos filtrar datos dentro de un **rango de valores**. Esto es útil cuando queremos seleccionar registros con valores entre un **mínimo** y un **máximo**. Por ejemplo, si estamos buscando productos con precios dentro de un rango específico, **BETWEEN** nos permite obtener todos los registros cuyo valor se encuentre entre esos dos límites, **incluyendo los valores extremos** (el mínimo y el máximo).

Instrucciones:

- Usa **BETWEEN** para seleccionar registros cuyo valor esté dentro de un rango de valores.
- **BETWEEN** incluye los valores límite. Esto significa que si tu rango es de 10 a 50, también se incluirán los registros cuyo valor sea 10 o 50.

Ejemplo:

Imaginemos que necesitamos obtener todos los productos cuyo **precio** esté entre **10 y 50**:

```
SELECT nombre, precio
```

```
FROM productos
```

```
WHERE precio BETWEEN 10 AND 50;
```

Esta consulta devolverá todos los productos cuyo precio esté **entre 10 y 50**, incluyendo esos valores extremos (10 y 50).

Usando IN para Seleccionar Varias Opciones

El operador **IN** es útil cuando queremos comprobar si un valor está **dentro de un conjunto de valores posibles**. En lugar de escribir varias condiciones usando **OR**, podemos usar **IN** para simplificar la consulta. Esto es útil cuando tenemos varias opciones y queremos seleccionar registros que coincidan con **al menos uno de esos valores**.

Instrucciones:

- Usa **IN** cuando tengas varios valores posibles y quieras seleccionar registros que coincidan con cualquiera de ellos.
- **IN** funciona con números, cadenas de texto, fechas y otros tipos de datos.

Ejemplo:

Supongamos que necesitamos obtener todos los empleados que trabajan en los departamentos de **Ventas**, **Marketing** o **Desarrollo**. Podríamos hacerlo de esta manera:

```
SELECT nombre, departamento
```

```
FROM empleados
```

```
WHERE departamento IN ('Ventas', 'Marketing', 'Desarrollo');
```

Esta consulta devolverá todos los empleados cuyo **departamento** sea **Ventas**, **Marketing** o **Desarrollo**, sin necesidad de escribir múltiples condiciones con **OR**.

Usando IS NULL para Filtrar Valores Nulos

En bases de datos, los valores **NULL** indican que **no hay valor asignado** a una columna. A veces, necesitamos filtrar registros con valores **NULL** en ciertas columnas, como cuando queremos encontrar productos sin fecha de vencimiento, por ejemplo.

Instrucciones:

- Usa **IS NULL** cuando quieras encontrar registros con valores **NULL** en una columna específica.
- Si necesitas buscar registros donde una columna **no sea NULL**, usa **IS NOT NULL**.

Ejemplo:

Imaginemos que queremos ver todos los productos que no tienen una **fecha de vencimiento** registrada (es decir, donde la columna **fecha_vencimiento** es **NULL**):

```
SELECT nombre, fecha_vencimiento
```

```
FROM productos
```

```
WHERE fecha_vencimiento IS NULL;
```

Esta consulta devolverá todos los productos donde la **columna fecha_vencimiento** no tiene ningún valor asignado, es decir, es **NULL**.

Usando LIKE para Búsquedas con Patrones (Cadenas de Texto)

El operador **LIKE** se utiliza para realizar **búsquedas con patrones** dentro de cadenas de texto. Esto permite realizar búsquedas más flexibles, buscando coincidencias parciales o específicas, usando **comodines**:

- %: Representa cualquier número de caracteres (incluso cero caracteres).
- _: Representa exactamente un carácter.

Instrucciones:

- Usa **LIKE** cuando quieras buscar patrones dentro de cadenas de texto.
- El comodín % reemplaza cualquier número de caracteres, y _ reemplaza solo un carácter.

Ejemplos:

1. Buscar productos cuyo nombre comience con "Cam":

Si queremos encontrar todos los productos cuyo nombre **empiece** con "Cam", usamos:

```
SELECT nombre
```

```
FROM productos
```

```
WHERE nombre LIKE 'Cam%';
```

Esto devolverá productos cuyo nombre empiece con "Cam", como **Cámara**, **Camisa**, **Camiseta**, etc.

2. Buscar productos cuyo nombre contenga "luz":

Si buscamos productos cuyo nombre **contenga** "luz" en cualquier parte, usamos:

```
SELECT nombre
```

```
FROM productos
```

```
WHERE nombre LIKE '%luz%';
```

Esto devolverá productos cuyo nombre tenga "luz" en cualquier parte del texto, como **Luz LED**, **Luz cálida**, etc.

3. Buscar productos cuyo nombre tenga exactamente tres letras:

Si necesitamos productos cuyo nombre tenga exactamente **tres caracteres**, podemos usar el comodín _:

```
SELECT nombre
```

```
FROM productos
```

```
WHERE nombre LIKE '___';
```

Esto devolverá productos cuyo nombre tenga exactamente tres caracteres.

Creación de Tablas en SQL

Cuando trabajamos con bases de datos, crear tablas es un paso esencial, ya que las tablas actúan como contenedores para almacenar los datos organizados. En SQL, usamos el comando **CREATE TABLE** para definir las tablas, sus columnas, los tipos de datos, y las restricciones que garantizarán la integridad de los datos.

Sintaxis Básica de CREATE TABLE

La estructura básica para crear una tabla incluye:

- **Nombre de la tabla:** Es el identificador que usaremos para referirnos a la tabla.
- **Columnas:** Cada columna tiene un nombre, un tipo de dato, y puede incluir restricciones opcionales.

Sintaxis General:

```
CREATE TABLE nombre_de_tabla (  
    nombre_columna1 tipo_de_dato restricción,  
    nombre_columna2 tipo_de_dato restricción,  
    ...  
);
```

Elementos Clave:

- **nombre_de_tabla:** Nombre que asignaremos a la tabla.
- **nombre_columnaX:** Los nombres de las columnas de la tabla.
- **tipo_de_dato:** Define el tipo de datos que almacenará cada columna (ejemplo: INT, VARCHAR, DATE, etc.).
- **restricción (opcional):** Reglas como NOT NULL, PRIMARY KEY, UNIQUE, o FOREIGN KEY para garantizar la integridad de los datos.

Ejemplo 1: Crear una Tabla Básica

Supongamos que necesitamos una tabla llamada **productos** para almacenar información básica sobre los productos de una tienda.

Instrucciones:

1. La columna `id_producto` será un número entero único para cada producto.

2. La columna nombre_producto almacenará el nombre del producto como texto, con un límite de 150 caracteres.
3. La columna precio almacenará el precio del producto como un número decimal.

SQL:

```
CREATE TABLE productos (  
    id_producto INT NOT NULL,  
    nombre_producto VARCHAR(150),  
    precio DECIMAL(10, 2)  
);
```

Detalles:

- id_producto INT NOT NULL: Define que los valores serán enteros y no pueden ser nulos.
- nombre_producto VARCHAR(150): Define una columna de texto con un límite de 150 caracteres.
- precio DECIMAL(10, 2): Almacena números con hasta 10 dígitos totales, de los cuales 2 son decimales.

Ejemplo 2: Agregar una Clave Primaria

Ahora queremos que la columna id_producto sea una clave primaria para garantizar que cada producto tenga un identificador único.

SQL:

```
CREATE TABLE productos (  
    id_producto INT NOT NULL PRIMARY KEY,  
    nombre_producto VARCHAR(150),  
    precio DECIMAL(10, 2)  
);
```

Detalles:

- PRIMARY KEY: Asegura que los valores en id_producto sean únicos y no nulos.

Ejemplo 3: Crear Relaciones con Claves Foráneas

Si tenemos dos tablas relacionadas, como **categorías** y **productos**, podemos usar una clave foránea para establecer una relación.

Escenario:

1. La tabla **categorías** tendrá una columna `id_categoria` como clave primaria.
2. La tabla **productos** tendrá una columna `id_categoria` que se relacionará con la columna `id_categoria` de la tabla **categorías**.

SQL para la tabla categorías:

```
CREATE TABLE categorias (  
    id_categoria INT NOT NULL PRIMARY KEY,  
    nombre_categoria VARCHAR(100) NOT NULL  
);
```

SQL para la tabla productos:

```
CREATE TABLE productos (  
    id_producto INT NOT NULL PRIMARY KEY,  
    nombre_producto VARCHAR(150),  
    precio DECIMAL(10, 2),  
    id_categoria INT,  
    FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)  
);
```

Ejemplo 4: Restringir Valores con UNIQUE

A veces queremos evitar duplicados en una columna que no es la clave primaria. Por ejemplo, en una tabla de empleados, el correo electrónico debe ser único.

SQL:

```
CREATE TABLE empleados (  
    id_empleado INT NOT NULL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    correo_electronico VARCHAR(100) UNIQUE
```

);

Detalles:

- correo_electronico VARCHAR(100) UNIQUE: Garantiza que los valores en esta columna no se repitan.

Ejemplo 5: Agregar Restricciones NOT NULL

La restricción **NOT NULL** asegura que una columna no pueda tener valores nulos. Por ejemplo, en una tabla de pedidos, necesitamos que las columnas fecha_pedido y total siempre tengan un valor.

SQL:

```
CREATE TABLE pedidos (  
    id_pedido INT NOT NULL PRIMARY KEY,  
    fecha_pedido DATE NOT NULL,  
    total DECIMAL(10, 2) NOT NULL  
);
```

Detalles:

- fecha_pedido DATE NOT NULL: La columna siempre debe tener una fecha válida.
- total DECIMAL(10, 2) NOT NULL: El total del pedido no puede ser nulo.

Actualizar Datos con SQL (UPDATE)

Al trabajar con bases de datos, una tarea común es modificar los datos existentes. Para hacerlo, SQL proporciona el comando **UPDATE**, que permite actualizar el contenido de una o más columnas en una tabla, según las condiciones que definamos. Es una herramienta poderosa que debe usarse con cuidado, ya que, si no se especifica correctamente, puede modificar más datos de los previstos.

¿Cómo Funciona el Comando UPDATE?

Para realizar una actualización, debemos:

1. **Elegir la tabla** que contiene los datos a modificar.
2. **Especificar qué columnas** y con qué valores queremos actualizarlas.

3. **Definir condiciones** para limitar qué filas se actualizarán (usando la cláusula WHERE).

Sintaxis General:

UPDATE nombre_de_tabla

SET columna1 = nuevo_valor1, columna2 = nuevo_valor2, ...

WHERE condición;

- **nombre_de_tabla**: La tabla que queremos modificar.
- **SET**: Define qué columnas cambiarán y sus nuevos valores.
- **WHERE** (opcional): Filtra las filas que se actualizarán. Si no se incluye, **todas las filas de la tabla** serán actualizadas.

Ejemplo 1: Actualizar un Solo Registro

Supongamos que tenemos una tabla **empleados** con las columnas id_empleado, nombre, y salario. Queremos actualizar el salario de un empleado específico.

Instrucciones:

1. Modifiquemos el salario del empleado con id_empleado = 1.
2. El nuevo salario será **3000**.

SQL:

UPDATE empleados

SET salario = 3000

WHERE id_empleado = 1;

Explicación:

- **SET salario = 3000**: Actualiza la columna salario con el valor 3000.
- **WHERE id_empleado = 1**: Asegura que solo se modifique la fila donde el id_empleado sea 1.

Ejemplo 2: Actualizar Múltiples Registros

Queremos aumentar el salario en un **10%** de todos los empleados que pertenecen al departamento **Ventas**.

SQL:

UPDATE empleados

SET salario = salario * 1.10

WHERE departamento = 'Ventas';

Explicación:

- **salario = salario * 1.10:** Incrementa el salario actual en un 10%.
- **WHERE departamento = 'Ventas':** Restringe la actualización a los empleados del departamento de Ventas.

Ejemplo 3: Actualizar Varias Columnas de una Fila

Si queremos actualizar más de una columna a la vez, podemos hacerlo en una sola consulta. Supongamos que necesitamos cambiar el **nombre** y el **salario** del empleado con `id_empleado = 3`.

Instrucciones:

1. Cambiemos el nombre del empleado a **Ana López**.
2. Establezcamos su nuevo salario en **4000**.

SQL:

UPDATE empleados

SET nombre = 'Ana López', salario = 4000

WHERE id_empleado = 3;

Explicación:

- **SET nombre = 'Ana López', salario = 4000:** Actualiza simultáneamente las columnas nombre y salario.
- **WHERE id_empleado = 3:** Limita la actualización al empleado con `id_empleado` igual a 3.

Ejemplo 4: Actualizar Todos los Registros

Si no usamos una condición (WHERE), el comando **UPDATE** afectará a **todas las filas de la tabla**. Por ejemplo, queremos establecer un salario uniforme de **5000** para todos los empleados.

SQL:

```
UPDATE empleados
```

```
SET salario = 5000;
```

Explicación:

- **No hay cláusula WHERE**, por lo que todos los registros serán modificados.
- Esto puede ser útil para ajustes globales, pero **es muy riesgoso si no se usa con cuidado**.

Ejemplo 5: Actualizar Basándose en Otra Tabla

En escenarios más avanzados, puede ser necesario actualizar datos basándonos en valores de otra tabla. Por ejemplo, tenemos dos tablas: **empleados** y **aumentos**. Queremos aumentar el salario de los empleados según los valores en la tabla **aumentos**.

Instrucciones:

1. La tabla **aumentos** tiene las columnas **id_empleado** y **aumento**.
2. Queremos incrementar el salario de cada empleado usando el valor de aumento correspondiente.

SQL:

```
UPDATE empleados
```

```
SET salario = salario + (
```

```
    SELECT aumento
```

```
    FROM aumentos
```

```
    WHERE aumentos.id_empleado = empleados.id_empleado
```

```
)
```

```
WHERE id_empleado IN (SELECT id_empleado FROM aumentos);
```

Explicación:

- **salario = salario + (...)**: Aumenta el salario actual con el valor obtenido de la subconsulta.
- **Subconsulta**: Encuentra el aumento correspondiente en la tabla **aumentos** para cada empleado.

- **WHERE id_empleado IN (...):** Limita la actualización solo a los empleados que tienen un registro en la tabla **aumentos**.

Eliminar Datos en SQL: DROP vs. DELETE vs. TRUNCATE

En SQL, a veces necesitamos eliminar datos o incluso tablas completas. Las tres principales herramientas para esto son **DROP**, **DELETE** y **TRUNCATE**. Aunque parecen similares, cada una tiene un propósito y comportamiento específicos. A continuación, se explica detalladamente cómo funcionan, cuándo utilizarlas y las diferencias clave entre ellas.

DROP: Elimina Toda la Tabla u Otro Objeto de la Base de Datos

La sentencia **DROP** elimina completamente una tabla (o cualquier otro objeto, como índices o vistas) de la base de datos. Esto significa que se borran tanto los datos como la estructura de la tabla, sin posibilidad de recuperación a menos que exista un respaldo.

Características:

- Elimina **toda la tabla** junto con sus datos y estructuras asociadas (índices, claves, etc.).
- **No es reversible:** Si se ejecuta, los datos y la tabla se pierden permanentemente.
- Es útil para eliminar objetos que ya no son necesarios.

Sintaxis:

```
DROP TABLE nombre_de_tabla;
```

Ejemplo:

Si queremos eliminar completamente una tabla llamada **empleados**, usamos:

```
DROP TABLE empleados;
```

Esto eliminará la tabla **empleados** y todo su contenido de la base de datos.

DELETE: Borra Filas Específicas de una Tabla

La sentencia **DELETE** se utiliza para eliminar **registros individuales o múltiples** de una tabla, sin afectar su estructura. Permite utilizar condiciones para seleccionar los registros que se desean borrar.

Características:

- Borra solo los datos que cumplen una condición definida con **WHERE**.

- Si no se especifica WHERE, elimina **todos los registros** de la tabla, pero la tabla sigue existiendo.
- Registra la operación en el log de transacciones, lo que permite revertirla dentro de una transacción.
- Puede activar **triggers** definidos en la tabla.

Sintaxis:

DELETE FROM nombre_de_tabla WHERE condición;

Ejemplo 1: Eliminar Registros Específicos

Queremos eliminar a los empleados con un salario menor a **2000**:

DELETE FROM empleados

WHERE salario < 2000;

Solo se eliminarán las filas que cumplan esta condición.

Ejemplo 2: Eliminar Todos los Registros

Si queremos eliminar todos los datos, pero conservar la estructura de la tabla:

DELETE FROM empleados;

Esto borra todos los registros, pero la tabla sigue estando disponible.

TRUNCATE: Borra Todos los Datos de una Tabla Rápidamente

TRUNCATE elimina **todos los registros** de una tabla de manera más rápida y eficiente que **DELETE**, pero conserva la estructura de la tabla. No permite especificar condiciones y restablece contadores como los de campos **AUTO_INCREMENT**.

Características:

- Borra todos los datos de la tabla, pero **mantiene su estructura**.
- No permite el uso de WHERE, ya que afecta a toda la tabla.
- No registra la operación en el log de transacciones, por lo que **no es reversible**.
- No activa triggers.
- Es ideal para limpiar tablas grandes de forma rápida.

Sintaxis:

TRUNCATE TABLE nombre_de_tabla;

Ejemplo:

Para eliminar todos los registros de la tabla **empleados**:

```
TRUNCATE TABLE empleados;
```

Esto borra todos los datos de la tabla, pero mantiene sus columnas, índices y otras configuraciones.

¿Cuándo Usar Cada Uno?

1. **Usar DROP:**

Cuando ya no necesitas la tabla ni sus datos. Es ideal para limpiar objetos obsoletos o para reconstruir una tabla desde cero.

2. **Usar DELETE:**

Cuando necesitas eliminar **registros específicos** o tener la opción de revertir los cambios en caso de error.

Ejemplo: Eliminar empleados con salario inferior a 2000.

3. **Usar TRUNCATE:**

Cuando necesitas **eliminar todos los registros** de una tabla rápidamente, pero deseas conservar su estructura para futuros usos.

Ejemplo: Limpiar todos los datos de la tabla empleados.

Modificación de Tablas en SQL: ALTER TABLE

Cuando trabajamos con bases de datos, es común necesitar cambios en la estructura de una tabla ya creada sin perder los datos almacenados. Para esto, usamos el comando **ALTER TABLE**, que permite realizar diversas modificaciones, como agregar columnas, cambiar tipos de datos, eliminar columnas, entre otras.

¿Qué Podemos Hacer con ALTER TABLE?

ALTER TABLE ofrece flexibilidad para modificar una tabla sin necesidad de eliminarla. Entre las operaciones más comunes están:

1. **Agregar nuevas columnas.**
2. **Eliminar columnas existentes.**
3. **Modificar el tipo de datos o las propiedades de una columna.**
4. **Renombrar columnas para hacerlas más descriptivas.**
5. **Cambiar el nombre de la tabla.**
6. **Añadir restricciones** (como claves primarias, foráneas o de unicidad).

Sintaxis Básica

El uso de **ALTER TABLE** sigue el siguiente patrón:

ALTER TABLE nombre_de_tabla [operación_a_realizar];

- **nombre_de_tabla:** Es el nombre de la tabla que queremos modificar.
- **operación_a_realizar:** Es el cambio que queremos hacer (agregar, modificar o eliminar una columna, cambiar nombres, etc.).

Operaciones Comunes con ALTER TABLE

1. Agregar una Columna a una Tabla

Si necesitas almacenar más información en una tabla, puedes agregar una nueva columna sin afectar los datos existentes.

Sintaxis:

ALTER TABLE nombre_de_tabla **ADD** nombre_columna tipo_de_dato;

Ejemplo:

Imagina que tienes una tabla **empleados** y quieres agregar la columna `fecha_ingreso` para registrar cuándo ingresó cada empleado a la empresa.

ALTER TABLE empleados **ADD** fecha_ingreso **DATE**;

Resultado: Se agrega la nueva columna `fecha_ingreso`. Los registros antiguos tendrán un valor **NULL** en esta columna hasta que se actualicen.

2. Eliminar una Columna

Cuando una columna ya no es necesaria, puedes eliminarla usando **DROP COLUMN**. Esto borra la columna y todos los datos almacenados en ella.

Sintaxis:

ALTER TABLE nombre_de_tabla **DROP COLUMN** nombre_columna;

Ejemplo:

Supongamos que la columna `edad` en la tabla **empleados** ya no se necesita.

ALTER TABLE empleados **DROP COLUMN** edad;

Resultado: La columna `edad` se elimina permanentemente.

3. Modificar el Tipo de Datos de una Columna

Si necesitas cambiar el tipo de datos de una columna (por ejemplo, de INT a DECIMAL), usa **MODIFY COLUMN**.

Sintaxis:

```
ALTER TABLE nombre_de_tabla MODIFY COLUMN nombre_columna  
nuevo_tipo_de_dato;
```

Ejemplo:

La columna salario fue definida como INT, pero ahora quieres incluir decimales.

```
ALTER TABLE empleados MODIFY COLUMN salario DECIMAL(10, 2);
```

Resultado: La columna salario ahora acepta valores decimales con 2 dígitos después del punto.

4. Renombrar una Columna

Para cambiar el nombre de una columna a algo más claro o descriptivo, usa **RENAME COLUMN**.

Sintaxis:

```
ALTER TABLE nombre_de_tabla RENAME COLUMN nombre_columna_actual TO  
nuevo_nombre_columna;
```

Ejemplo:

Renombrar la columna nombre_empleado a nombre_completo en la tabla **empleados**:

```
ALTER TABLE empleados RENAME COLUMN nombre_empleado TO  
nombre_completo;
```

Resultado: La columna ahora se llamará nombre_completo.

5. Renombrar una Tabla

Puedes cambiar el nombre de toda la tabla si su propósito o contexto ha cambiado.

Sintaxis:

```
ALTER TABLE nombre_tabla_actual RENAME TO nuevo_nombre_tabla;
```

Ejemplo:

Si la tabla **empleados** ahora almacena datos generales de trabajadores, puedes renombrarla:

ALTER TABLE empleados RENAME TO trabajadores;

Resultado: La tabla ahora se llama **trabajadores**.

6. Agregar Restricciones (Constraints)

Las restricciones ayudan a mantener la integridad de los datos en la base de datos. Puedes agregar restricciones como claves primarias, foráneas o de unicidad.

Agregar una Clave Primaria

Sintaxis:

ALTER TABLE nombre_de_tabla ADD CONSTRAINT nombre_constraint PRIMARY KEY (columna);

Ejemplo:

Establecer la columna id_empleado como clave primaria en la tabla **empleados**:

ALTER TABLE empleados ADD CONSTRAINT pk_empleados PRIMARY KEY (id_empleado);

Resultado: La columna id_empleado ahora asegura que todos sus valores sean únicos y no nulos.

Agregar una Clave Foránea

Sintaxis:

ALTER TABLE nombre_de_tabla ADD CONSTRAINT nombre_constraint FOREIGN KEY (columna) REFERENCES otra_tabla(otra_columna);

Ejemplo:

Relacionar la columna id_departamento de la tabla **empleados** con la columna id_departamento de la tabla **departamentos**:

ALTER TABLE empleados ADD CONSTRAINT fk_departamento FOREIGN KEY (id_departamento) REFERENCES departamentos(id_departamento);

Resultado: La base de datos asegura que los valores en id_departamento de **empleados** coincidan con los de **departamentos**.

Operaciones Comunes con Varias Tablas en SQL

Cuando trabajamos con bases de datos relacionales, a menudo necesitamos combinar información de varias tablas. SQL nos proporciona diferentes métodos para hacerlo, dependiendo de lo que queremos lograr.

Uso de JOIN para Combinar Tablas

¿Qué es un JOIN?

- JOIN combina datos de dos o más tablas basándose en una relación común entre ellas (normalmente una clave foránea o primaria).

Tipos de JOIN:

1. INNER JOIN

- Devuelve solo las filas que tienen coincidencias en ambas tablas.
- Es el tipo de JOIN más común.

Sintaxis:

SELECT columnas

FROM tabla1

INNER JOIN tabla2

ON tabla1.columna = tabla2.columna;

Ejemplo:

Si tenemos:

- Una tabla empleados con columnas id_empleado, nombre, id_departamento.
- Una tabla departamentos con columnas id_departamento, nombre_departamento.

Queremos obtener el nombre de los empleados junto con su departamento:

SELECT empleados.nombre, departamentos.nombre_departamento

FROM empleados

INNER JOIN departamentos

ON empleados.id_departamento = departamentos.id_departamento;

Resultado: Solo se muestran los empleados que tienen un departamento asignado.

LEFT JOIN (LEFT OUTER JOIN)

Devuelve todas las filas de la tabla de la izquierda y las coincidencias de la tabla de la derecha. Si no hay coincidencias, muestra NULL para las columnas de la tabla de la derecha.

Ejemplo:

Queremos listar todos los empleados, incluso si no tienen un departamento asignado:

```
SELECT empleados.nombre, departamentos.nombre_departamento
```

```
FROM empleados
```

```
LEFT JOIN departamentos
```

```
ON empleados.id_departamento = departamentos.id_departamento;
```

- Los empleados sin un departamento tendrán NULL en la columna nombre_departamento.

RIGHT JOIN (RIGHT OUTER JOIN)

Devuelve todas las filas de la tabla de la derecha y las coincidencias de la tabla de la izquierda. Si no hay coincidencias, muestra NULL en las columnas de la tabla de la izquierda.

Ejemplo:

Queremos listar todos los departamentos, incluso si no tienen empleados asignados:

```
SELECT empleados.nombre, departamentos.nombre_departamento
```

```
FROM empleados
```

```
RIGHT JOIN departamentos
```

```
ON empleados.id_departamento = departamentos.id_departamento;
```

- Los departamentos sin empleados tendrán NULL en las columnas de la tabla empleados.

FULL JOIN (FULL OUTER JOIN)

Devuelve todas las filas de ambas tablas, mostrando NULL donde no haya coincidencias.

Ejemplo:

Queremos listar todos los empleados y todos los departamentos, aunque no tengan coincidencias:

```
SELECT empleados.nombre, departamentos.nombre_departamento
```

```
FROM empleados
```

```
FULL JOIN departamentos
```

```
ON empleados.id_departamento = departamentos.id_departamento;
```

CROSS JOIN

Genera el producto cartesiano de dos tablas (todas las combinaciones posibles entre las filas de ambas tablas). Úsalo con precaución, ya que puede generar un número muy alto de registros.

Ejemplo:

Si tenemos una tabla colores y otra tamaños:

```
SELECT colores.color, tamaños.tamaño
```

```
FROM colores
```

```
CROSS JOIN tamaños;
```

- Resultado: Todas las combinaciones posibles entre colores y tamaños.

Uso de UNION para Combinar Resultados de Consultas

¿Qué es UNION?

- Combina los resultados de dos o más consultas.
- Las consultas deben tener el mismo número de columnas y tipos de datos compatibles.

1. UNION:

- Elimina duplicados.

Ejemplo:

Tenemos tablas empleados_2023 y empleados_2024. Queremos una lista combinada sin duplicados:

```
SELECT nombre FROM empleados_2023
```

```
UNION
```

```
SELECT nombre FROM empleados_2024;
```

2. UNION ALL:

- Incluye duplicados.

Ejemplo:

```
SELECT nombre FROM empleados_2023
```

```
UNION ALL
```

```
SELECT nombre FROM empleados_2024;
```

Expresiones SQL: CASE y COALESCE

CASE:

Permite realizar operaciones condicionales dentro de las consultas.

Ejemplo: Clasificar empleados según su salario:

```
SELECT nombre, salario,  
  
       CASE  
  
         WHEN salario > 5000 THEN 'Alto'  
  
         WHEN salario BETWEEN 3000 AND 5000 THEN 'Medio'  
  
         ELSE 'Bajo'  
  
       END AS categoria_salario  
  
FROM empleados;
```

COALESCE:

Devuelve el primer valor no nulo de una lista de opciones.

Ejemplo: Mostrar "No disponible" si un número de teléfono es NULL:

```
SELECT nombre,  
  
       COALESCE(telefono, 'No disponible') AS telefono  
  
FROM empleados;
```

Filtrando Datos Agrupados (HAVING)

Cuando trabajas con datos agrupados en SQL mediante la cláusula GROUP BY, la cláusula HAVING te permite filtrar esos datos agrupados según ciertos criterios. Esto es especialmente útil para trabajar con funciones agregadas como SUM, AVG, COUNT, etc.

Diferencia entre WHERE y HAVING:

1. **WHERE:** Filtra los datos **antes** de que se agrupen. No puede usarse con funciones agregadas.
2. **HAVING:** Filtra los datos **después** de que se han agrupado. Es compatible con funciones agregadas.

Ejemplo:

Imagina que tienes una tabla ventas con las columnas:

- producto: Nombre del producto.
- cantidad: Número de unidades vendidas.
- precio: Precio por unidad.

Si deseas mostrar solo los productos cuya venta total (cantidad * precio) supere los \$5000, necesitas usar HAVING:

```
SELECT producto, SUM(cantidad * precio) AS total_ventas  
FROM ventas  
GROUP BY producto  
HAVING SUM(cantidad * precio) > 5000;
```

Explicación del proceso:

1. **GROUP BY:** Agrupa las filas por producto.
2. **SUM:** Calcula el total de ventas para cada producto.
3. **HAVING:** Filtra solo los productos donde el total de ventas supera \$5000.

Cláusula OVER y Funciones de Ventana

La cláusula OVER permite realizar cálculos avanzados sin necesidad de agrupar datos. Esto es útil para generar estadísticas, promedios acumulados, clasificaciones, y más, sin perder los datos originales.

Estructura:

```
SELECT columna, función_de_ventana() OVER (PARTITION BY columna2 ORDER BY  
columna3)  
FROM tabla;
```

- **PARTITION BY:** Divide los datos en grupos lógicos similares a GROUP BY, pero sin consolidarlos en una sola fila.
- **ORDER BY:** Especifica el orden en el que se aplicará la función dentro de cada grupo.

Ejemplo: Promedio acumulado:

Si deseas calcular el promedio acumulado de ventas de cada producto según la fecha de venta:

SELECT producto, fecha, cantidad,

AVG(cantidad) OVER (PARTITION BY producto ORDER BY fecha) AS
promedio_acumulado

FROM ventas;

Explicación:

1. **PARTITION BY producto:** Agrupa las filas por producto.
2. **ORDER BY fecha:** Ordena las filas de cada producto por fecha.
3. **AVG(cantidad):** Calcula el promedio acumulado de las cantidades para cada producto.

Funciones de Ventana:

1. **FIRST_VALUE y LAST_VALUE:**
 - **FIRST_VALUE:** Devuelve el primer valor de un grupo.
 - **LAST_VALUE:** Devuelve el último valor de un grupo.

Ejemplo: Obtener la primera venta de cada producto:

SELECT producto, fecha, cantidad,

FIRST_VALUE(cantidad) OVER (PARTITION BY producto ORDER BY fecha) AS
primer_venta

FROM ventas;

2. **LAG y LEAD:**

- **LAG:** Devuelve el valor de la fila anterior.
- **LEAD:** Devuelve el valor de la fila siguiente.

Ejemplo: Obtener la cantidad vendida el día anterior para cada producto:

SELECT producto, fecha, cantidad,

LAG(cantidad) OVER (PARTITION BY producto ORDER BY fecha) AS
venta_anterior

FROM ventas;

Clasificación de Registros Agrupados: RANK y ROW_NUMBER

1. **ROW_NUMBER**: Asigna un número único a cada fila dentro de un grupo.

Ejemplo: Enumerar las ventas de cada producto en orden de mayor a menor cantidad:

```
SELECT producto, cantidad,
```

```
    ROW_NUMBER() OVER (PARTITION BY producto ORDER BY cantidad DESC)  
    AS numero_fila
```

```
FROM ventas;
```

2. **RANK**: Asigna un rango basado en el orden especificado. Si hay empates, las filas tienen el mismo rango, pero se salta el siguiente número.

Ejemplo: Clasificar productos según el total de ventas:

```
SELECT producto, SUM(cantidad * precio) AS total_ventas,
```

```
    RANK() OVER (ORDER BY SUM(cantidad * precio) DESC) AS ranking  
FROM ventas
```

```
GROUP BY producto;
```

Vistas (Views)

Las vistas son consultas guardadas que pueden ser tratadas como tablas. Son útiles para simplificar consultas complejas y mejorar la legibilidad.

Crear una Vista:

```
CREATE VIEW vista_ventas AS
```

```
SELECT producto, SUM(cantidad * precio) AS total_ventas
```

```
FROM ventas
```

```
GROUP BY producto;
```

Ahora puedes usar esta vista como si fuera una tabla:

```
SELECT * FROM vista_ventas WHERE total_ventas > 5000;
```

Procedimientos Almacenados (Stored Procedures)

Un procedimiento almacenado es un conjunto de instrucciones SQL que puedes ejecutar cuando sea necesario.

Ejemplo: Actualizar el salario de un empleado:

```
CREATE PROCEDURE actualizar_salario(id_empleado INT, nuevo_salario DECIMAL)
AS
BEGIN
    UPDATE empleados
    SET salario = nuevo_salario
    WHERE id_empleado = id_empleado;
END;

Para ejecutarlo:

EXEC actualizar_salario 1, 5000;
```

Disparadores (Triggers)

Un disparador se activa automáticamente al ocurrir ciertos eventos (INSERT, UPDATE, DELETE).

Ejemplo: Historial de cambios de salario:

```
CREATE TRIGGER historial_salario
AFTER UPDATE ON empleados
FOR EACH ROW
BEGIN
    INSERT INTO historial_salarios (id_empleado, salario_anterior, salario_nuevo)
    VALUES (OLD.id_empleado, OLD.salario, NEW.salario);
END;
```

Administración de Bases de Datos en SQL

La administración de bases de datos implica una serie de tareas clave para garantizar que los datos estén organizados, seguros y disponibles cuando se necesiten

1. Crear una Base de Datos

¿Qué es una base de datos?

Una base de datos es un espacio organizado dentro de un servidor donde se almacenan tablas, vistas, procedimientos y otros objetos. Crear una base de datos es el primer paso para comenzar a gestionar datos.

Comando para crear una base de datos:

La instrucción CREATE DATABASE se utiliza para crear una nueva base de datos en el servidor SQL.

Sintaxis:

```
CREATE DATABASE nombre_base_de_datos;
```

Ejemplo práctico: Imagina que estás configurando una base de datos para una tienda. El comando sería:

```
CREATE DATABASE tienda;
```

¿Qué sucede después de ejecutarlo?

- Se crea un nuevo espacio llamado tienda dentro del servidor.
- Ahora puedes añadir tablas, vistas y otros objetos en esta base de datos.

2. Respalidar una Base de Datos

¿Por qué hacer respaldos?

Los respaldos son esenciales para proteger los datos frente a posibles fallos en el sistema, corrupción de archivos o errores humanos. El respaldo genera una copia de la base de datos en un archivo externo.

Comando para respaldar:

La instrucción BACKUP DATABASE crea un archivo de respaldo.

Sintaxis:

```
BACKUP DATABASE nombre_base_de_datos
```

```
TO DISK = 'ruta_del_archivo.bak';
```

Ejemplo práctico: Respalidar la base de datos tienda en un archivo llamado tienda_backup.bak:

```
BACKUP DATABASE tienda
```

```
TO DISK = 'C:\backups\tienda_backup.bak';
```

3. Restaurar una Base de Datos

¿Cuándo necesitas restaurar una base de datos?

Si los datos se pierden, se corrompen o necesitas regresar a una versión anterior, puedes restaurar la base de datos utilizando un archivo de respaldo.

Comando para restaurar:

La instrucción RESTORE DATABASE recupera una base de datos desde un archivo .bak.

Sintaxis:

```
RESTORE DATABASE nombre_base_de_datos
```

```
FROM DISK = 'ruta_del_archivo.bak';
```

Ejemplo práctico: Restaurar la base de datos tienda desde el archivo tienda_backup.bak:

```
RESTORE DATABASE tienda
```

```
FROM DISK = 'C:\backups\tienda_backup.bak';
```

Importar y Exportar Archivos

Importar y exportar datos son operaciones comunes al interactuar con fuentes externas como hojas de cálculo o sistemas de terceros. Estas tareas se realizan con herramientas específicas:

a) Importar datos con BULK INSERT

Este comando permite cargar grandes cantidades de datos desde archivos como CSV o de texto plano a una tabla en SQL.

Sintaxis:

```
BULK INSERT nombre_tabla
```

```
FROM 'ruta_del_archivo'
```

```
WITH (
```

```
    FIELDTERMINATOR = 'separador_campos',
```

```
    ROWTERMINATOR = 'separador_filas'
```

```
);
```

Ejemplo práctico: Importar datos de un archivo CSV llamado clientes.csv a la tabla clientes:

```
BULK INSERT clientes
```

```
FROM 'C:\datos\clientes.csv'
```

```
WITH (
```

```
    FIELDTERMINATOR = ',', -- Los campos están separados por comas.
```

```
    ROWTERMINATOR = '\n' -- Cada fila está separada por un salto de línea.
```

```
);
```

b) Exportar datos con bcp (Bulk Copy Program)

El programa bcp es una herramienta de línea de comandos que permite exportar datos desde una base de datos SQL Server a un archivo externo.

Sintaxis básica:

```
bcp nombre_base_de_datos.dbo.nombre_tabla out 'ruta_del_archivo'
```

```
-S servidor -U usuario -P contraseña -c
```

Ejemplo práctico: Exportar los datos de la tabla clientes a un archivo CSV:

```
bcp tienda.dbo.clientes out 'C:\datos\clientes_exportados.csv'
```

```
-S mi_servidor -U mi_usuario -P mi_contraseña -c
```

¿Qué hace este comando?

- Exporta los datos de la tabla clientes a un archivo en formato CSV.
- Usa -c para especificar que el formato es de texto plano (carácter).

Consideraciones Finales para la Administración de Bases de Datos

1. Planifica respaldos periódicos:

- Define una estrategia de respaldo basada en la criticidad de los datos.
- Realiza pruebas de restauración periódicas para asegurarte de que los respaldos funcionan.

2. Automatiza procesos:

- Usa trabajos programados (como SQL Server Agent) para respaldos, restauraciones e importaciones.

3. Gestiona permisos:

- Asegúrate de que solo los usuarios autorizados tengan acceso a comandos de respaldo y restauración.

4. Monitorea el almacenamiento:

- Revisa regularmente el espacio en disco para respaldos y datos. Una falta de espacio puede causar problemas serios.

Modelado de una Base de Datos Relacional en SQL

El modelado de una base de datos relacional es el proceso de estructurar y organizar los datos de manera lógica para garantizar eficiencia, integridad y facilidad de uso.

Elementos Clave del Modelado Relacional

1. Modelo Entidad-Relación (ER)
2. Llaves (Primarias y Secundarias)
3. Restricciones (Constraints)
4. Índices
5. Normalización

1. Modelo Entidad-Relación (ER)

El modelo ER es una representación gráfica de los datos y sus interrelaciones. Es esencial para planificar cómo se estructurarán los datos en una base de datos relacional.

- **Entidades:** Representan objetos o conceptos importantes (por ejemplo, *Clientes*, *Productos*).
- **Relaciones:** Representan cómo interactúan las entidades entre sí (por ejemplo, *Compra* o *Realiza*).
- **Atributos:** Son las propiedades de una entidad (por ejemplo, *nombre* o *email* de un cliente).

Ejemplo: Base de datos de una tienda en línea

- **Entidades:**
 - *Clientes* (atributos: *id_cliente*, *nombre*, *email*).

- *Productos* (atributos: id_producto, nombre, precio).
- **Relación:**
 - *Realiza:* Conecta *Clientes* con *Productos* mediante un pedido.

En un **diagrama ER**, las entidades se representan como rectángulos y las relaciones como líneas que las conectan.

2. Llaves (Primarias y Secundarias)

Las llaves son fundamentales para identificar registros y establecer relaciones entre tablas.

Llave Primaria (Primary Key)

- Identifica de manera única cada registro en una tabla.
- Reglas:
 - No puede contener valores duplicados.
 - No puede ser nula.

Ejemplo: Tabla de Clientes

```
CREATE TABLE Clientes (  
    id_cliente INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    email VARCHAR(100)  
);
```

Aquí, id_cliente es la llave primaria.

Llave Secundaria (Foreign Key)

- Vincula registros entre dos tablas estableciendo relaciones.
- Define cómo una tabla hace referencia a la llave primaria de otra.

Ejemplo: Relación entre Pedidos y Clientes

```
CREATE TABLE Pedidos (  
    id_pedido INT PRIMARY KEY,  
    fecha DATE,  
    id_cliente INT,
```

FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)

);

- id_cliente en *Pedidos* es una llave secundaria que se refiere a id_cliente en *Clientes*.
- Esto asegura que cada pedido esté asociado a un cliente válido.

3. Restricciones (Constraints)

Las restricciones son reglas que controlan qué valores son válidos en las tablas, ayudando a mantener la integridad de los datos.

Tipos de Restricciones:

- **DEFAULT:** Asigna un valor predeterminado a una columna si no se especifica otro valor.
- CREATE TABLE Pedidos (
 - id_pedido INT PRIMARY KEY,
 - estado VARCHAR(50) DEFAULT 'pendiente'
-);

Si no se especifica el estado, tomará el valor 'pendiente'.

- **NOT NULL:** Asegura que una columna no pueda tener valores nulos.
- CREATE TABLE Clientes (
 - id_cliente INT PRIMARY KEY,
 - nombre VARCHAR(100) NOT NULL
-);

Aquí, nombre debe contener siempre un valor.

- **CHECK:** Valida que los valores de una columna cumplan una condición específica.
- CREATE TABLE Productos (
 - id_producto INT PRIMARY KEY,
 - precio DECIMAL(10, 2) CHECK (precio > 0)
-);

Esto impide que se inserten precios negativos o nulos.

- **UNIQUE:** Garantiza que los valores de una columna sean únicos.
- CREATE TABLE Clientes (
- email VARCHAR(100) UNIQUE
-);

Ningún cliente puede tener el mismo correo electrónico.

4. Índices

Los índices son estructuras de datos que aceleran las búsquedas en las tablas. Sin embargo, pueden ralentizar las operaciones de escritura como inserciones y actualizaciones.

Crear un Índice:

```
CREATE INDEX idx_email ON Clientes (email);
```

Esto mejora el rendimiento de las consultas que busquen clientes por email.

5. Normalización

La normalización organiza los datos para:

- Reducir la redundancia.
- Eliminar dependencias innecesarias.

Formas Normales:

Primera Forma Normal (1NF):

- Los datos deben ser atómicos (indivisibles).
- Cada columna debe contener un único valor.

Ejemplo:

Una tabla que almacena varios productos por pedido en una sola celda es incorrecta:

id_pedido productos

1	"Producto A, Producto B"
---	--------------------------

Debe dividirse en:

id_pedido producto

1	Producto A
1	Producto B

Segunda Forma Normal (2NF):

- Debe estar en 1NF.
- No debe haber dependencias parciales (atributos que dependan solo de parte de la llave primaria).

Ejemplo:

En una tabla que incluye id_cliente y nombre_cliente junto a pedidos:

id_pedido id_cliente nombre_cliente producto

1	101	Juan Pérez	Producto A
---	-----	------------	------------

El nombre del cliente depende del id_cliente, no del id_pedido.

Se separa en:

1. Tabla *Clientes*:
2. CREATE TABLE Clientes (
3. id_cliente INT PRIMARY KEY,
4. nombre_cliente VARCHAR(100)
5.);
6. Tabla *Pedidos*:
7. CREATE TABLE Pedidos (
8. id_pedido INT PRIMARY KEY,
9. id_cliente INT,
10. producto VARCHAR(100),
11. FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
12.);

Tercera Forma Normal (3NF):

- Debe estar en 2NF.
- Elimina dependencias transitivas (atributos no clave que dependen de otros atributos no clave).

Ejemplo:

Si una tabla incluye ciudad_cliente, que depende de id_cliente:

id_pedido	id_cliente	ciudad_cliente	producto
1	101	Madrid	Producto A

Debe separarse:

1. Tabla *Ciudades*:
2. CREATE TABLE Ciudades (
3. id_cliente INT PRIMARY KEY,
4. ciudad_cliente VARCHAR(100)
5.);
6. Tabla *Pedidos*: Relaciona clientes con productos.

Optimización de Consultas SQL

- **Uso de EXPLAIN o EXPLAIN PLAN:**

Estas herramientas te permiten ver cómo se ejecuta una consulta, lo que te ayuda a identificar cuellos de botella en el rendimiento. Muestran el orden en que se procesan las tablas, qué índices se usan, y qué operaciones están tardando más.

Ejemplo: Si una consulta utiliza un JOIN entre dos tablas grandes, EXPLAIN puede mostrarte si el índice está siendo utilizado correctamente o si la consulta está haciendo un escaneo completo de las tablas (lo que es ineficiente).

- **Identificación y solución de cuellos de botella:**

Las consultas ineficientes pueden ser identificadas por su uso excesivo de recursos. Puedes mejorar estas consultas reescribiendo su lógica, usando índices apropiados o simplificando las subconsultas.

Manejo de índices avanzados

Índices compuestos:

Son índices creados sobre varias columnas de una tabla, lo que permite mejorar el rendimiento en consultas que filtran por más de una columna. Es útil cuando las consultas suelen filtrar por una combinación de campos, no solo por uno.

Ejemplo: Si tienes una tabla de Pedidos y consultas frecuentemente por `id_cliente` y `fecha`, un índice compuesto en esas dos columnas puede acelerar esas búsquedas.

Índices únicos vs. no únicos:

Los índices únicos no solo mejoran el rendimiento, sino que también aseguran que no haya duplicados en las columnas indexadas (esto es diferente de las restricciones `UNIQUE`).

Los índices no únicos aceleran las búsquedas sin imponer una restricción de unicidad.

Particionado de Tablas

- **Estrategias de partición:**

El particionado de tablas divide grandes conjuntos de datos en partes más pequeñas (particiones). Esto puede ser por rango (fechas, por ejemplo), lista (categorías específicas), o hash (distribución aleatoria de datos).

Ejemplo: Una tabla de ventas puede ser particionada por año, donde cada partición contiene solo los datos de un año en particular.

- **Ventajas en rendimiento:**

Las consultas que filtran por el campo de partición se benefician del particionado porque solo acceden a una partición, no a la tabla completa.

Gestión de datos históricos

- **Tablas de archivo (archival tables):**

A medida que los datos crecen, almacenar todo en una única tabla puede hacer que las consultas sean más lentas. Las tablas de archivo permiten mover datos antiguos a una tabla separada, manteniendo el rendimiento de la tabla principal.

Seguridad y Control de Acceso

Gestión de usuarios y permisos

- **Creación y control de accesos:**

Es fundamental definir qué usuarios pueden hacer qué en la base de datos. Esto se logra creando roles y asignando permisos de lectura, escritura, y administración a esos roles.

Ejemplo: Puedes tener un rol ADMIN con acceso total y un rol USER solo con permisos de lectura.

- **Uso de roles y esquemas:**

Los roles agrupan permisos que luego se asignan a los usuarios, facilitando la gestión de permisos. Los esquemas permiten organizar objetos de la base de datos, y se usan para definir la visibilidad y el acceso de diferentes grupos de usuarios.

Cifrado de datos

- **Cifrado en columnas:**

El cifrado ayuda a proteger la información sensible, como contraseñas o datos bancarios. El cifrado puede aplicarse a columnas específicas en lugar de cifrar toda la base de datos.

Ejemplo: Al almacenar contraseñas, es común cifrarlas con una clave de cifrado antes de guardarlas en la base de datos.

- **Protección de información sensible:**

Usar métodos como el cifrado SSL/TLS para las conexiones entre clientes y bases de datos garantiza que la información no sea interceptada durante su transmisión.

Auditoría

- **Uso de triggers para auditoría:**

Los triggers son eventos que se ejecutan automáticamente cuando se realiza una acción en la base de datos. Pueden usarse para registrar cambios en tablas sensibles (como cuando un usuario actualiza un precio o elimina un registro).

Gestión Avanzada de Transacciones

Bloqueos y concurrencia

- **Tipos de bloqueos:**

Los bloqueos impiden que múltiples usuarios modifiquen los mismos datos simultáneamente. Los bloqueos pueden ser de lectura (lectura compartida) o de escritura (bloqueo exclusivo).

- **Niveles de aislamiento de transacciones:**

Los niveles de aislamiento determinan cómo se manejan las transacciones concurrentes:

- **READ UNCOMMITTED:** Permite leer datos no confirmados (puede generar lecturas sucias).
- **READ COMMITTED:** Solo lee datos confirmados.
- **REPEATABLE READ:** Garantiza que no haya cambios en los datos leídos durante la transacción.
- **SERIALIZABLE:** El nivel más alto, donde las transacciones son completamente aisladas entre sí.

Manejo de deadlocks

- **Identificación y evitación:**

Un "deadlock" ocurre cuando dos o más transacciones se bloquean mutuamente. Se pueden identificar a través de los logs del sistema o las herramientas de diagnóstico de la base de datos.

La prevención incluye diseñar transacciones cortas y con un orden fijo de acceso a los recursos.

Temas Relacionados con Big Data y SQL Moderno

Integración con sistemas NoSQL

- **Acceso a datos no relacionales desde SQL:**

Muchas bases de datos modernas como PostgreSQL o SQL Server permiten almacenar datos JSON o XML. Esto facilita la integración entre bases de datos relacionales y NoSQL sin perder la capacidad de realizar consultas SQL.

- **Tablas híbridas con columnas tipo JSON o ARRAY:**

Algunas bases de datos permiten tener columnas con tipos de datos más flexibles como JSON o arrays. Esto es útil cuando se necesita almacenar estructuras de datos no tabulares dentro de una base de datos relacional.

SQL para analítica

- **Funciones analíticas avanzadas:**

Las funciones analíticas permiten hacer cálculos sobre un conjunto de filas mientras mantienen las filas individuales intactas. Ejemplo: LEAD, LAG, RANK, ROW_NUMBER son funciones que se utilizan para análisis de series temporales y ordenación de datos.

SQL distribuido

- **Bases de datos distribuidas:**

Algunas plataformas de Big Data, como Apache Hive o Google BigQuery, permiten ejecutar SQL sobre datos distribuidos a través de múltiples servidores. Esto facilita el

análisis de grandes volúmenes de datos sin necesidad de mover toda la información a una única máquina.

Buenas Prácticas y Estilo de Escritura

Estandarización de consultas

- **Convenciones de nombres:**

Es importante definir una convención para nombrar tablas, columnas y otros objetos en la base de datos, para que el código sea legible y mantenible.

Ejemplo: Usar nombres en plural para las tablas (clientes en lugar de cliente), y nombres claros para las columnas (fecha_nacimiento en lugar de fn).

Gestión de scripts

- **Control de versiones para SQL:**

Utilizar herramientas como Git para manejar versiones de scripts SQL. Esto facilita la colaboración y el seguimiento de cambios en el esquema y las consultas.

Gestión de entornos de desarrollo y producción

- **Separación de entornos:**

Tener entornos separados para desarrollo, pruebas y producción asegura que los cambios no afecten al sistema en producción y permite probar consultas de manera segura.

Herramientas Complementarias

Exploradores de bases de datos

- **Uso de herramientas gráficas:**

Herramientas como SQL Server Management Studio (SSMS), pgAdmin o DBeaver permiten a los usuarios interactuar con las bases de datos de manera más visual, facilitando la creación, consulta y gestión de bases de datos.

Automatización con scripts

- **Uso de herramientas como cron o PowerShell:**

Las tareas repetitivas, como ejecutar backups o limpiar tablas, pueden automatizarse usando scripts. Herramientas como cron en Linux o Task Scheduler en Windows permiten ejecutar estos scripts a intervalos regulares.

Consideraciones para Diferentes Motores de Bases de Datos

Diferencias entre SQL estándar y SQL en sistemas específicos

- **MySQL, PostgreSQL, SQL Server y Oracle:**

Cada sistema tiene características y sintaxis particulares. Por ejemplo, MySQL no soporta funciones de ventana de la misma forma que PostgreSQL, y SQL Server tiene funciones adicionales como TOP y IDENTITY para gestionar la creación de registros únicos.